Assignment Report
Course: Advanced Python (ICS0019)
Team members: Radions Laškovs, Ilja Priimak
Date: 22.05.2026
Repository link: GitHub

1. Approach

1.1 Strategy Overview

Our overall strategy was focused on improving detection of minority attack classes (R2L and U2R), because the baseline models handled Normal and DoS traffic relatively well but almost completely failed on rare attacks. Major part of the experiments focused on handling class imbalance using SMOTE, SMOTEENN, custom class weights, and probability threshold tuning. After tree-based models stopped showing significant improvement and remained around 0.60 macro F1-score, we shifted our focus toward Keras neural networks.

We also experimented with feature engineering, feature selection, stacking ensembles, voting ensembles, threshold tuning, and categorical embeddings.

1.2 Preprocessing

- **Feature engineering:** Several additional features were created during experiments. Traffic ratio and interaction-based features such as byte ratios, combinations of connection statistics that could better separate minority attack classes from normal traffic. Feature engineering mainly targeted improving R2L and U2R detection.
- **Feature selection:** Feature selection was tested in later experiments. Low-importance and redundant features were removed based on XGBoost feature importance scores. Encoded feature set was reduced from 121 features to approximately 61 features in some experiments. This slightly improved generalization and reduced noise.
- **Scaling:** StandardScaler was applied for models sensitive to feature magnitude, especially SVM and neural network experiments. Tree-based models such as Random Forest, XGBoost, and LightGBM were mostly trained without scaling because they are less sensitive to feature scales.
- **Other:** Categorical features were encoded as numerical values using label encoding. In later experiments, we also tested neural-network embeddings for categorical features instead of traditional encoded representations. Random seeds were fixed in most experiments to improve reproducibility, although neural network experiments still showed noticeable instability between runs.

1.3 Class Imbalance Handling

Because NSL-KDD is highly imbalanced, especially for U2R attacks, handling imbalance became the main focus.

- **Method used:** SMOTE oversampling; SMOTEENN hybrid resampling; class_weight='balanced'; Manually tuned custom class weights; Sample weighting; Threshold/probability tuning; Ensemble-based balancing approaches.
- **Parameters:** Most SMOTE experiments used default k_neighbors=5 configuration, while later experiments manually adjusted minority target sizes for R2L and U2R. Neural network experiments used aggressively tuned custom class weights to force the model to pay more attention to minority attack classes. Some experiments also used threshold multipliers to increase minority-class prediction probability.
- **Effect on training set distribution:** SMOTE and SMOTEENN significantly increased number of minority-class samples, especially for R2L and U2R, creating much more balanced training distribution. This improved recall for rare attacks but sometimes reduced precision and introduced instability or overfitting. Custom class weights achieved better balance without directly modifying the dataset and produced the best overall results in later Keras experiments.

2. Experiments

Total number of experiments: 37

Experiments Summary:

#	Description	Algorithm	Imbalance Handling	Macro F1 (CV)	Macro F1 (test)
1	Random Forest baseline	Random Forest	class_weight='balanced' tested	0.9126 ± 0.0371	0.4707
2	SMOTE + Random Forest	Random Forest	SMOTE	0.9369 ± 0.0269	0.5497
3	SMOTE + XGBoost	XGBoost	SMOTE	0.9437 ± 0.0262	0.6073
5	SMOTE + LightGBM	LightGBM	SMOTE	0.9296 ± 0.0283	0.5922
9	SMOTEENN + Tuned XGBoost	XGBoost	SMOTEENN	0.9542 ± 0.0291	0.6097
12	MLP Neural Network	MLP	None / SMOTE tested	0.8481 ± 0.0151	0.5996
13	Soft voting ensemble + probability tuning	Voting Ensemble	Mixed model-level handling	0.9415 ± 0.0258	0.6232
16	Keras neural network with custom weights	Keras NN	Custom class weights	0.8675 ± 0.0381	0.6399
19	Keras class weight + seed tuning	Keras NN	Tuned custom class weights	0.8525 ± 0.0197	0.6646
22	Keras categorical embeddings MLP	Keras NN with embeddings	Custom class weights	0.8321 ± 0.0506	0.6578
32	k-NN with scaling and SMOTE	k-NN	SMOTE	0.7641 ± 0.0076	0.6358
37	Keras + NN	Keras NN	Used custom values from Keras	0.7536 ± 0.0164	0.7139

3. Final Results

3.1 Best Model

- **Algorithm:** Keras + NN w. balanced weights
- **Key parameters:** Dropout: 0.25, Learning Rate: 0.00001, Epochs: 140, Patience: 12,
- **Imbalance handling:** {0: 1.0, 1: 0.7, 2: 2.0, 3: 20.0, 4: 60.0}
- **Feature engineering:** Changed fix() from shuffle=True (Default) to False. Gave stable reulsts but also random record scores. Used best score Keras weights as custom weights for this model.

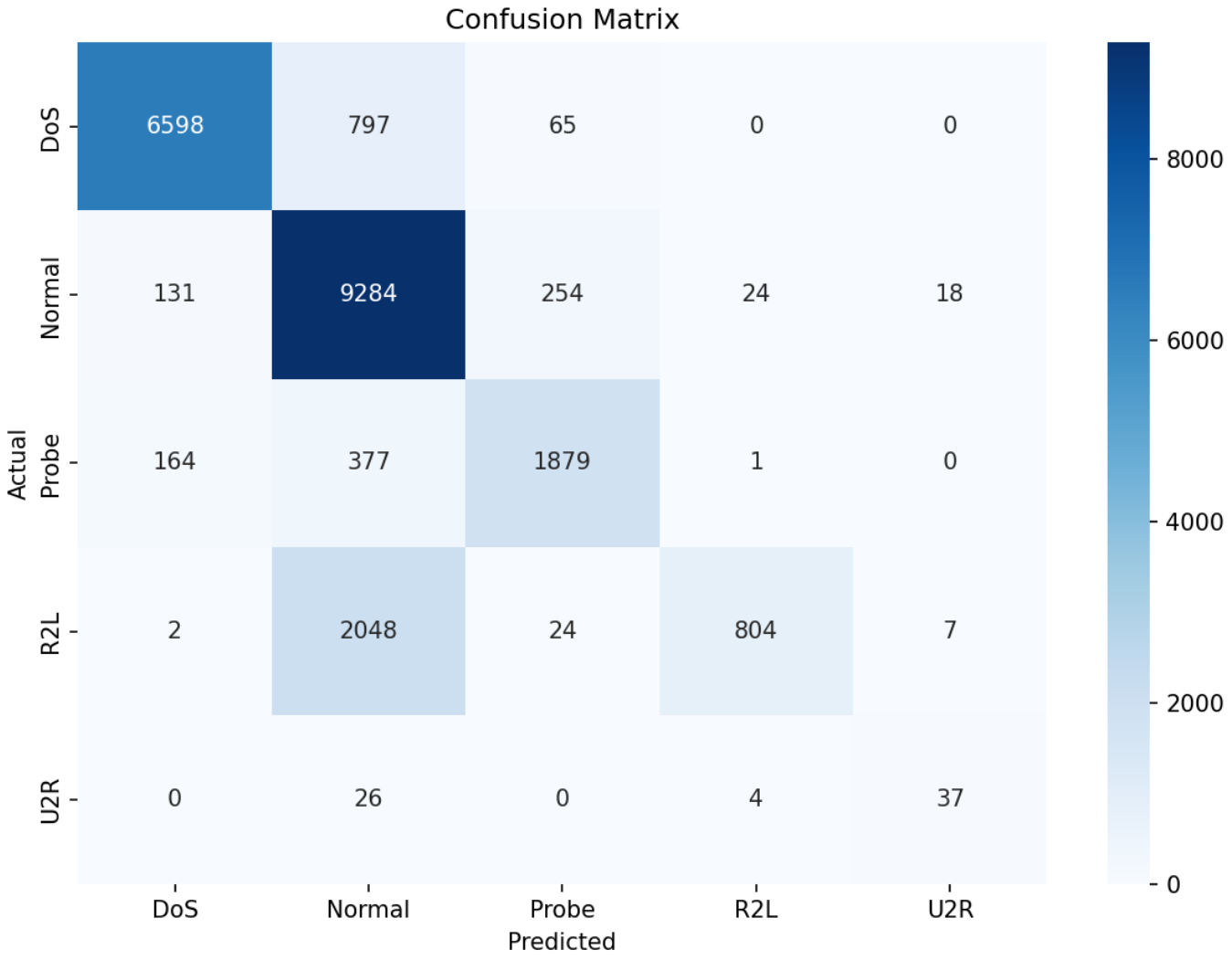
3.2 Final Macro F1-Score

Metric	Score
Macro F1 (test)	0.7139
Macro F1 (CV)	0.7536 ± 0.0164

3.3 Classification Report

Category	Precision	Recall	F1-Score	Support
Normal	0.74	0.96	0.83	9711
DoS	0.96	0.88	0.92	7460
Probe	0.85	0.78	0.81	2421
R2L	0.97	0.28	0.43	2885
U2R	0.60	0.55	0.57	67

3.4 Confusion Matrix



4. Cross-Validation vs. Test Score

- **CV macro F1:** 0.7536 ± 0.0164
- **Test macro F1:** 0.7139
- **Gap:** 0.0397

Analysis: Test macro F1-score is lower than cross-validation score by 0.0397, which is expected gap. Cross-validation was performed on KDDTrain+, while KDDTest+ contains different distribution and includes attack types that were not present during training. Model had to generalize to unseen attack patterns instead of only recognizing known examples.

Gap does not indicate overfitting, because difference between CV and test performance is not very large. Model still generalized reasonably well on test set and achieved strong performance on Normal, DoS, and Probe classes. Main weakness was R2L detection: many R2L samples were still classified as Normal, which reduced macro F1-score. U2R performance was much better than in the early experiments, but this class remained unstable because it has only 67 samples in the test set.

5. What Worked and What Didn't

What had the biggest positive impact?

- Biggest positive impact came from moving from tree-based models to Keras neural networks with carefully tuned class weights. Earlier models such as XGBoost, LightGBM, SMOTEENN, and voting ensembles improved over

Random Forest baseline, but most of them stayed around 0.60–0.62 test macro F1. Neural network experiments showed higher potential, especially after tuning class weights for R2L and U2R.

- Custom and balanced class weights helped model pay more attention to rare attack categories. R2L and U2R were the main reason why earlier models had low macro F1.

What surprisingly didn't help?

- Increasing neural network layer sizes did not bring clear improvement. For example, using larger hidden layers such as 256/128/64 gave similar results to smaller architectures such as 128/64/32.
- SMOTE helped tree-based models, especially XGBoost, but it did not work equally well for neural networks. In the MLP experiments, SMOTE actually reduced test macro F1-score because minority-class precision dropped.
- Threshold tuning also did not consistently help: for XGBoost, the best probability multipliers stayed at 1.0, and for Keras it made the model less stable.
- Ensemble methods were disappointing. Soft voting and stacking produced strong cross-validation scores, but they did not generalize well to KDDTest+.

What would you try with more time?

- With more time, we would focus more on neural networks, because they produced the best final results and showed highest potential.
- We would also continue experimenting with categorical embeddings, because embedding-based Keras model achieved one of the strongest test results and improved R2L detection. Better embedding architecture or a hybrid model combining embeddings with carefully tuned numerical features could improve performance further.
- Another direction could be more careful per-class tuning for R2L and U2R. Test separate validation strategies/probability calibration/two-stage model that first separates normal traffic from suspicious traffic and then classifies attack types.

Appendix: Environment

- **Hardware:** Intel i5-10600K, 64GB, 2xRTX2080Ti
- **Python version:** 3.12.3
- **Key libraries:** { absl-py==2.4.0 astunparse==1.6.3 certifi==2026.5.20 charset-normalizer==3.4.7 contourpy==1.3.3 cycler==0.12.1 flatbuffers==25.12.19 fonttools==4.63.0 gast==0.7.0 google-pasta==0.2.0 grpcio==1.80.0 h5py==3.14.0 idna==3.16 imbalanced-learn==0.14.1 joblib==1.5.3 keras==3.14.1 kiwisolver==1.5.0 libclang==18.1.1 lightgbm==4.6.0 Markdown==3.10.2 markdown-it-py==4.2.0 MarkupSafe==3.0.3 matplotlib==3.10.9 mdurl==0.1.2 ml_dtypes==0.5.4 namex==0.1.0 numpy==2.4.6 nvidia-cublas-cu12==12.9.2.10 nvidia-cuda-cupti-cu12==12.9.79 nvidia-cuda-nvcc-cu12==12.9.86 nvidia-cuda-nvrtc-cu12==12.9.86 nvidia-cuda-runtime-cu12==12.9.79 nvidia-cudnn-cu12==9.22.0.52 nvidia-cufft-cu12==11.4.1.4 nvidia-curand-cu12==10.3.10.19 nvidia-cusolver-cu12==11.7.5.82 nvidia-cusparse-cu12==12.5.10.65 nvidia-nccl-cu12==2.30.4 nvidia-nvjitlink-cu12==12.9.86 opt_einsum==3.4.0 optree==0.19.1 packaging==26.2 pandas==3.0.3 pillow==12.2.0 protobuf==7.35.0 Pygments==2.20.0 pyparsing==3.3.2 python-dateutil==2.9.0.post0 requests==2.34.2 rich==15.0.0 scikit-learn==1.8.0 scipy==1.17.1 seaborn==0.13.2 setuptools==82.0.1 six==1.17.0 sklearn-compat==0.1.5 tensorboard==2.20.0 tensorboard-data-server==0.7.2 tensorflow==2.20.0 termcolor==3.3.0 threadpoolctl==3.6.0 typing_extensions==4.15.0 urllib3==2.7.0 Werkzeug==3.1.8 wheel==0.47.0 wrapt==2.2.0 xgboost==3.2.0 }
- **Random seed:** 100 (highest results in overall)